

Python - Pandas – 2

Data Cleaning Using Pandas

Real-world datasets are rarely clean. Data collected from surveys, apps, schools, or businesses often contains missing values, repeated records, inconsistent text, and wrong data types.

Data cleaning is the process of fixing these problems so the data becomes reliable for analysis.

Why preprocessing is necessary:

- Incorrect data leads to incorrect results
- Clean data improves model performance
- Most data science work involves cleaning, not analysis

Example:

A university stores student information. Some students forget to enter age, some names are in uppercase, and some records appear twice. Pandas helps clean this data efficiently.

Handling Missing Values

Missing values occur when some data entries are empty or unavailable. Pandas represents missing values using NaN, None, or pd.NA.

Common reasons for missing data:

- User skips a form field
- Data entry error
- Sensor or system failure

```
import pandas as pd
```

```
data = {  
    "Name": ["Amit", "Riya", "Karan", "Sneha"],  
    "Age": [20, None, 22, None],  
    "Marks": [85, 90, None, 88]  
}  
  
df = pd.DataFrame(data)
```

Checking for Missing Data

Before cleaning, we must **identify where data is missing**.

- `isnull()` checks each cell
- Returns True for missing values
- `sum()` counts missing values per column

Example

```
df.isnull()
```

```
df.isnull().sum()
```

Here,

If Age shows 2 missing values, it means two students did not provide their age.

Dropping Missing Data

If missing data is very small or unimportant, it can be removed.

- Rows with missing values can be deleted
- Columns with too many missing values can be removed
- Useful when missing data does not affect analysis much

Example

```
df.dropna()
```

```
df.dropna(axis=1)
```

Here,

If only a few students skipped marks, their records can be removed to avoid confusion.

Filling Missing Data

Instead of removing data, missing values can be **replaced with meaningful values**.

Common strategies:

- Replace with 0
- Replace with average value
- Use previous or next values

Code Examples

```
df["Marks"].fillna(df["Marks"].mean())
```

```
df.fillna(0)
```

```
df.ffill()
```

```
df.bfill()
```

Here,

If average marks is 45, missing marks can be replaced with 45 to keep all student records.

Detecting & Removing Duplicates

Duplicate records occur when the same data is entered more than once.

```
data = {  
    "Student": ["Rahul", "Neha", "Rahul", "Aman"],  
    "Class": ["10A", "10B", "10A", "10A"]  
}
```

```
df = pd.DataFrame(data)
```

```
df
```

- `df.duplicated()` identifies repeated rows
- First occurrence is not considered a duplicate
- `df.drop_duplicates()` removes repeated entries

Example

```
df.duplicated()
```

```
df.drop_duplicates()
```

Here,

If Rahul from class 10A appears twice, one record should be removed to avoid double counting.

Duplicate Check Using Specific Columns

Sometimes duplication depends only on selected columns.

Example

```
df.duplicated(subset=["Student", "Class"])
```

Here,

If two rows have the same student name and class, they represent the same student.

String Operations Using .str

Text data is often inconsistent due to mixed casing, extra spaces, or different formats. The .str accessor allows string operations on entire columns.

```
data = {  
    "Name": [" Ankit ", "pooja", "RAHUL"],  
    "City": ["Delhi", "mumbai", "DELHI"]  
}
```

```
df = pd.DataFrame(data)
```

```
df
```

Examples

```
df["Name"].str.strip()
```

```
df["Name"].str.upper()
```

```
df["City"].str.contains("delhi", case=False)
```

```
df["Name"].str.split()
```

Type Conversion Using .astype() and pd.to_datetime()

Sometimes data is stored in the wrong format, such as numbers stored as text or dates stored as strings.

Examples

```
df["Age"] = df["Age"].astype(int)
```

```
df["JoinDate"] = pd.to_datetime(df["JoinDate"])
```

Why `pd.to_datetime()` is powerful:

- Handles multiple date formats
- Manages missing dates
- Converts timestamps automatically

Different date formats are converted into one standard format for easy comparison.

Applying Functions to Data

Custom logic can be applied to data using functions.

Using `.apply()`

```
df["Grades"] = df["Marks"].apply(lambda mark: "Good" if mark >= 90 else "Better")
```

Using `.map()`

```
grade_map = {"A": "Excellent", "B": "Good", "C": "Average"}
```

```
df["Performance"] = df["Grade"].map(grade_map)
```

Short codes are converted into meaningful labels.

Using `.replace()`

```
df["City"].replace({"cr": "Credit", "dr": "Debit"})
```

Abbreviated names are replaced with full names.

Data Transformation

Once data cleaning is completed, the next step is **data transformation**.

Data transformation involves **reshaping, reformatting, sorting, and reordering data** so that it is ready for analysis, reporting, or visualization. Pandas provides several flexible functions to perform these tasks efficiently.

Sorting and Ranking

Sorting helps arrange data in a logical order, making patterns and comparisons easier to identify.

Sorting by Values (sort_values())

The sort_values() function is used to sort a DataFrame based on one or more column values.

Sorting in Ascending Order

```
df.sort_values("Age")
```

- Sorts values from smallest to largest.
- Ascending order is the default behavior.

Example:

If the ages are 22, 18, 25, the sorted order will be 18, 22, 25.

Sorting in Descending Order

```
df.sort_values("Age", ascending=False)
```

- Sorts values from largest to smallest.

Example:

If salaries are 30000, 50000, 40000, the result will be 50000, 40000, 30000.

Sorting by Multiple Columns

```
df.sort_values(["Age", "Salary"])
```

- The DataFrame is first sorted by **Age**.
- If two or more rows have the same Age, they are then sorted by **Salary**.

Example:

Employees with the same age are ordered based on their salary, so higher earners appear first or last depending on the sort order.

Resetting the Index (reset_index())

After sorting or deleting rows, the index may no longer be in a proper sequence.

```
df.reset_index(drop=True, inplace=True)
```

- drop=True removes the old index.
- inplace=True updates the original DataFrame.

Example:

An index like 0, 4, 7 becomes 0, 1, 2 after resetting.

Sorting by Index (sort_index())

```
df.sort_index()
```

- Sorts the DataFrame based on index values.
- Useful when the index order has changed due to row operations.

Example:

If the index order is 3, 1, 5, it will be rearranged to 1, 3, 5.

Ranking Data (rank())

The rank() function assigns ranks to numerical values such as marks, scores, or sales figures.

Default Ranking Method

```
df["Rank"] = df["Score"].rank()
```

- Tied values receive the **average rank**.
- Can result in decimal values.

Example:

Scores: 88, 92, 92

Ranks: 3, 1.5, 1.5

Dense Ranking

```
df["Rank"] = df["Score"].rank(method="dense")
```

- Tied values get the same rank.
- No gaps in ranking numbers.

Example:

Scores: 95, 90, 90, 85

Ranks: 1, 2, 2, 3

This method is useful for exam results or leaderboard-style rankings.

Renaming Columns and Index

Renaming improves clarity and makes the DataFrame easier to understand.

Renaming Specific Columns

```
df.rename(columns={"oldName": "newName"}, inplace=True)
```

Example:

Renaming dob to Date_of_Birth for better readability.

Renaming Index Labels

```
df.rename(index={0: "row1", 1: "row2"}, inplace=True)
```

Example:

Useful when rows represent categories instead of numbers.

Renaming All Columns

```
df.columns = ["Name", "Age", "City"]
```

- The number of new names must match the number of columns.

Example:

After importing raw data with unnamed columns, meaningful names are assigned.

Changing Column Order

Column order can be changed to match reporting or analysis needs.

Reordering Columns Manually

```
df = df[["City", "Name", "Age"]]
```

- Columns appear in the exact order specified.

Example:

Location information can be moved to the front for regional analysis.

Moving One Column to the Front

```
cols = ["Name"] + [col for col in df.columns if col != "Name"]
```

```
df = df[cols]
```

- Keeps all columns intact.
- Moves a single important column to the first position.

Example:

Placing Student_Name at the beginning for mark sheets.

Reading & Writing Files in Pandas

Pandas provides powerful and easy-to-use functions to **read data from files** and **write DataFrames back to files**. These functions follow a simple naming pattern:

Working with CSV Files

CSV (Comma-Separated Values) files are one of the most commonly used formats for storing tabular data.

Reading CSV Files

To load a CSV file into Pandas, the `read_csv()` function is used. It converts the file into a DataFrame that can be easily analyzed and manipulated.

Common options while reading CSV files:

- Select only specific columns instead of loading the entire file

- Limit the number of rows to speed up loading for large datasets
- Handle missing values or different separators

Example:

```
import pandas as pd
```

```
students = pd.read_csv("students.csv", usecols=["RollNo", "Marks"], nrows=5)
```

Reads only the *RollNo* and *Marks* columns

Loads just the first 5 rows, improving performance

Writing CSV Files

After processing data, you may want to save it back to a CSV file. This is done using the `to_csv()` method.

Key points:

- By default, Pandas writes the DataFrame index as a column
- You can disable this if the index is not meaningful

Example:

```
students.to_csv("filtered_students.csv", index=False)
```

Saves clean data without adding an extra index column

Working with Excel Files

Excel files are widely used in business and reporting environments. Pandas supports both reading and writing Excel files efficiently.

Reading Excel Files

The `read_excel()` function is used to load data from Excel files.

Important options:

- Read a specific worksheet using `sheet_name`

- Load multiple sheets at once
- Work with large Excel files selectively

Example:

```
sales_data = pd.read_excel("company_data.xlsx", sheet_name="2024_Sales")
```

Reads data only from the *2024_Sales* worksheet

Writing Excel Files

To export a DataFrame to Excel, Pandas uses the `to_excel()` method.

Example:

```
sales_data.to_excel("annual_sales.xlsx", index=False)
```

Writes data without including the DataFrame index

Writing to Multiple Sheets in One Excel File

Sometimes reports require multiple sheets in a single Excel workbook. Pandas handles this using `ExcelWriter`.

Example:

```
with pd.ExcelWriter("analysis_report.xlsx") as writer:
```

```
    summary_df.to_excel(writer, sheet_name="Overview", index=False)
```

```
    detail_df.to_excel(writer, sheet_name="Full_Data", index=False)
```

Creates one Excel file

Stores different DataFrames in separate sheets

Useful for dashboards and management reports

Merging & Joining Data in Pandas

In real-world data analysis, information is often spread across **multiple tables or files**. Pandas provides flexible tools to **combine datasets**, similar to SQL joins, but with more control and options. The most commonly used methods are:

- `pd.merge()`
- `pd.concat()`
- `.join()`

Choosing the right method depends on **how the data is related** and **how you want it combined**.

SQL-Style Joins Using `pd.merge()`

The `pd.merge()` function is used when datasets share a **common key column**, similar to joins in SQL.

Inner Join (Default)

An inner join returns **only the rows where the key exists in both DataFrames**.

Key characteristics:

- Rows without matching keys are discarded
- Default behavior of `pd.merge()`

Example:

```
orders = pd.DataFrame({  
    "OrderID": [101, 102, 103],  
    "CustomerID": [1, 2, 4]  
})
```

```
customers = pd.DataFrame({  
    "CustomerID": [1, 2, 3],  
    "CustomerName": ["Aman", "Neha", "Ravi"]  
})
```

```
result = pd.merge(orders, customers, on="CustomerID")
```

Only orders with valid customer information are retained.

Left Join

A left join keeps **all rows from the left DataFrame** and fills missing values with NaN when no match is found in the right DataFrame.

Use case:

- When the left table is primary and must not lose rows

Example:

```
result = pd.merge(orders, customers, on="CustomerID", how="left")
```

Orders without a matching customer still appear, but customer details are missing.

Right Join

A right join keeps **all rows from the right DataFrame**, even if there is no matching key in the left DataFrame.

Use case:

- When the right table represents the complete reference list

Example:

```
result = pd.merge(orders, customers, on="CustomerID", how="right")
```

Customers without orders are included with missing order details.

Outer Join

An outer join returns **all rows from both DataFrames**.

Key points:

- Includes every key from both tables
- Missing values are filled with NaN

Example:

```
result = pd.merge(orders, customers, on="CustomerID", how="outer")
```

This is useful for full data audits and completeness checks.

Concatenating DataFrames Using pd.concat()

While merge() combines data based on keys, pd.concat() **stacks DataFrames together** either by rows or columns.

Vertical Concatenation (Row-wise)

Used when multiple DataFrames have the **same structure** and need to be stacked one below another.

Key points:

- Default behavior of pd.concat()
- Index values are preserved unless reset

Example:

```
week1 = pd.DataFrame({"Sales": [200, 300]})
```

```
week2 = pd.DataFrame({"Sales": [250, 400]})
```

```
combined = pd.concat([week1, week2])
```

This is useful for appending monthly or weekly data.

Horizontal Concatenation (Column-wise)

Used to combine **different features side-by-side**.

Important consideration:

- Index alignment matters
- Misaligned indexes result in NaN

Example:

```
student_info = pd.DataFrame({"StudentID": [1, 2, 3]})
```

```
scores = pd.DataFrame({"Marks": [85, 90, 78]})
```

```
combined = pd.concat([student_info, scores], axis=1)
```

Each row is combined based on index position.

Using .join() Method

The .join() method is used when:

- One DataFrame needs to be joined using its **index**
- The other DataFrame has a matching index or key

Example:

```
profiles = pd.DataFrame(  
    {"Age": [21, 22]},  
    index=["S1", "S2"]  
)
```

```
grades = pd.DataFrame(  
    {"Grade": ["A", "B"]},  
    index=["S1", "S2"]  
)
```

```
result = profiles.join(grades)
```

This is cleaner than merge() when indexes are already aligned.

Reshaping Data in Pandas

Data reshaping means **changing the structure of a DataFrame** without changing the actual data values. In Pandas, reshaping is commonly done to move between:

- **Wide format** → more columns, fewer rows
- **Long format** → fewer columns, more rows

Two key functions used for this purpose are:

- `melt()` → wide to long
- `pivot()` → long to wide

These operations are essential for **data analysis, visualization, and statistical modeling**.

`melt()` — Converting Wide Data to Long Data

The `melt()` function is used to **unpivot** a DataFrame. It converts multiple columns into **row-wise key-value pairs**, producing a tidy, long-format dataset.

When to Use `melt()`

Use `melt()` when:

- Each row represents one entity (such as a person or product)
- Multiple columns represent different measurements or categories
- You want a format suitable for plotting or grouping operations

Key Parameters of `melt()`

- `id_vars`
Columns that remain unchanged and act as identifiers
- `value_vars`
Columns that are converted into rows
- `var_name`
Name of the new column that stores original column names
- `value_name`
Name of the new column that stores the values
- `col_level`
Used when working with multi-level column indexes

Example of melt()

```
import pandas as pd
```

```
data = {  
    "Employee": ["Rahul", "Sneha", "Amit"],  
    "Q1": [12000, 15000, 13000],  
    "Q2": [14000, 16000, 13500],  
    "Q3": [15500, 17000, 14500]  
}
```

```
df = pd.DataFrame(data)
```

```
long_df = df.melt(  
    id_vars=["Employee"],  
    value_vars=["Q1", "Q2", "Q3"],  
    var_name="Quarter",  
    value_name="Revenue"  
)
```

Explanation

- "Employee" stays fixed as an identifier
- Quarterly columns are converted into rows
- Each row now represents **one employee-quarter combination**

Why Use melt()

- Helps in **data normalization**
- Makes datasets compatible with **visualization libraries**
- Converts columns into rows for **tidy data formats**
- Simplifies grouping and aggregation operations

pivot() — Converting Long Data to Wide Data

The pivot() function performs the opposite of melt(). It reshapes long-format data into a **wide-format table** by spreading categorical values across columns.

How pivot() Works

pivot() requires three components:

- Rows (index)
- Columns (columns)
- Values (values)

Each unique combination of index and column creates a cell in the new DataFrame.

Key Parameters of pivot()

- index
Column whose unique values become row labels
- columns
Column whose unique values become new column headers
- values
Column containing the data to fill the table

Example of pivot()

```
data = {  
    "Employee": ["Rahul", "Rahul", "Sneha", "Sneha", "Amit", "Amit"],  
    "Quarter": ["Q1", "Q2", "Q1", "Q2", "Q1", "Q2"],  
    "Revenue": [12000, 14000, 15000, 16000, 13000, 13500]  
}
```

```
df = pd.DataFrame(data)

wide_df = df.pivot(

    index="Employee",
    columns="Quarter",
    values="Revenue"

)
```

Explanation

- Employees become rows
- Quarters become columns
- Revenue values fill the table cells

Why Use pivot()

- Creates a **clean, readable structure**
- Makes comparisons across categories easier
- Ideal for **reports and summary tables**
- Useful before exporting data to Excel

Handling Duplicate Values with pivot_table()

A limitation of pivot() is that it **cannot handle duplicate index-column pairs**. If duplicates exist, Pandas raises an error.

In such cases, use pivot_table(), which allows **aggregation**.

Example Using pivot_table()

```
data = {

    "Product": ["Pen", "Pen", "Notebook", "Notebook"],
    "Month": ["Jan", "Jan", "Jan", "Feb"],
    "Sales": [100, 120, 200, 180]

}
```

```
df = pd.DataFrame(data)
```

```
summary = df.pivot_table(  
    index="Product",  
    columns="Month",  
    values="Sales",  
    aggfunc="mean"  
)
```

Explanation

- Duplicate entries are aggregated using the mean
- Missing combinations appear as NaN
- Supports functions like sum, mean, count, and max

Aggregation & Grouping in Pandas

Aggregation and grouping allow you to **summarize large datasets** and answer analytical questions.

Pandas makes this easy using the `.groupby()` method combined with aggregation functions.

The `.groupby()` Function

The `.groupby()` method splits data into groups based on one or more columns, applies a function to each group, and then combines the results.

Conceptually, it follows three steps:

1. **Split** the data into groups
2. **Apply** an aggregation or transformation
3. **Combine** the results into a new DataFrame or Series

Basic Grouping with Single Column

When grouping by one column, Pandas calculates results **per unique value** in that column.

Example

```
avg_salary = df.groupby("Department")["Salary"].mean()
```

Explanation:

- Data is grouped by department
- Average salary is calculated for each department
- Output contains one row per department

Aggregate Functions

Common Aggregation Functions

Frequently used aggregation functions include:

- `mean()` → average value
- `sum()` → total value
- `count()` → number of records
- `min()` → smallest value
- `max()` → largest value
- `median()` → middle value

These functions help summarize numeric data efficiently.

Examples

```
df.groupby("Team")["Salary"].mean() # Average salary per team
```

```
df.groupby("Team")["Salary"].sum() # Total salary per team
```

```
df.groupby("Team")["Salary"].count() # Number of employees per team
```

```
df.groupby("Team")["Salary"].min() # Minimum salary per team
```

```
df.groupby("Team")["Salary"].max() # Maximum salary per team
```

Each aggregation gives a **different insight** from the same dataset.

Grouping by Multiple Columns

You can group data using more than one column to get **more granular summaries**.

Example

```
df.groupby(["Department", "Gender"])["Salary"].mean()
```

Explanation:

- First groups by department
- Then splits each department by gender
- Calculates average salary for each subgroup

This is useful for comparisons such as salary differences across genders within departments.

Custom Aggregations with .agg()

The .agg() (or .aggregate()) method allows:

- Applying **multiple aggregation functions at once**
- Using **custom names** for output columns
- Applying **custom functions**

Note: .agg() and .aggregate() are identical; they are aliases.

Applying Multiple Aggregations

```
df.groupby("Department")["Salary"].agg(["mean", "min", "max"])
```

This returns multiple summary statistics for each department in one step.

Using .transform()

The .transform() method performs a group-wise operation **but returns output with the same shape as the original DataFrame**.

When to Use .transform()

- When you want to compare each row to its group
- When the result must align row-by-row with original data

Example of .transform()

```
df["Dept_Avg_Salary"] = df.groupby("Department")["Salary"].transform("mean")
```

Explanation:

- Computes average salary per department
- Assigns that value to every employee in the same department
- Original number of rows is preserved

This is commonly used for normalization and comparisons.

Using .filter()

The .filter() method is used to **keep or remove entire groups** based on a condition.

When to Use .filter()

- When you want to exclude groups that don't meet certain criteria
- When decisions are based on group-level statistics

Example of .filter()

```
filtered_df = df.groupby("Department").filter(  
    lambda x: x["Salary"].mean() > 100  
)
```

Explanation:

- Calculates mean salary for each department
- Keeps only departments with an average salary above 85
- All rows of qualifying departments are retained